

# Módulo RMSE

Respecto a un valor ideal

---

**Responsable:** JOscar Vladimir Nij Cruz

**Archivo:** modulo\_1\_rmse.s

**Biblioteca común:** utils.s

**Resultado:** resultado\_rmse.txt

**Arquitectura:** ARM64

## Índice

|                                |   |
|--------------------------------|---|
| 1. Algoritmo                   | 2 |
| 2. Flujo de rutina             | 2 |
| 3. Registros utilizados        | 3 |
| 4. Ciclos, saltos y subrutinas | 4 |
| 5. Memoria                     | 6 |
| 6. Entrada y salida            | 6 |
| 7. Utilización de utils.s      | 7 |
| 8. Compilación y ejecución     | 7 |

## 1 Algoritmo

El módulo implementa el cálculo RMSE, o raíz del error cuadrático medio, respecto a un valor ideal. La rutina analiza una columna específica del archivo `lecturas.csv` dentro de una ventana definida por una línea inicial y una línea final.

El módulo recibe cuatro parámetros:

```
inicio fin columna ideal
```

Las columnas válidas para el cálculo son:

| Número | Columna             |
|--------|---------------------|
| 2      | TEMP                |
| 3      | HUM_AIRE            |
| 4      | HUM_SUELO_1 / SOIL1 |
| 6      | LUZ                 |
| 7      | GAS                 |

No se calcula RMSE sobre ID, columnas de riego ni sobre HUM\_SUELO\_2. Esta validación evita procesar columnas que no representan una serie de sensor útil para el análisis histórico.

## Cálculos implementados

$$ERROR_i = Y_i - IDEAL$$

$$ERROR2_i = ERROR_i \times ERROR_i$$

$$MSE = \frac{\sum ERROR2_i}{N}$$

$$RMSE = \sqrt{MSE}$$

Todas las operaciones se realizan con enteros. La división es entera truncada y la raíz cuadrada se calcula mediante una rutina de raíz entera.

## 2 Flujo de rutina

La rutina inicia leyendo los argumentos desde la pila. Luego convierte los parámetros numéricos, valida la columna, valida el rango solicitado y calcula la cantidad esperada de datos.

Después llama a la función `read_column_to_stack` de `utils.s`. Esta función lee el archivo `lecturas.csv`, extrae la columna solicitada dentro del rango indicado y guarda los valores en la pila.

El módulo compara la cantidad de datos leídos contra la cantidad esperada. Si no coinciden, se considera un rango inválido. Si la cantidad de datos es menor a dos, se devuelve error de datos insuficientes.

Cuando las validaciones son correctas, se recorre la lista de valores, se calcula el error respecto al ideal, se eleva al cuadrado, se acumula la suma, se divide entre la cantidad de datos y finalmente se calcula la raíz cuadrada entera.

### 3 Registros utilizados

| Registro | Uso dentro del módulo                                                                                                                         |
|----------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| x9       | Guarda la cantidad de argumentos recibidos desde la pila.                                                                                     |
| x0       | Registro de paso de parámetros y retorno de funciones como <code>cadena_a_entero</code> , <code>integer_sqrt</code> y conversiones numéricas. |
| x1       | Dirección de cadenas o buffers utilizados para impresión y escritura.                                                                         |
| x2       | Longitud de cadenas al imprimir o escribir archivos. También recibe la cantidad de datos retornada por <code>read_column_to_stack</code> .    |
| x8       | Número de syscall para escritura y salida del programa.                                                                                       |
| x10      | Valor actual leído desde la pila durante el recorrido de datos.                                                                               |
| x11      | Columna seleccionada enviada hacia <code>read_column_to_stack</code> .                                                                        |
| x12      | Registro auxiliar para cargar direcciones de variables en memoria.                                                                            |
| x13      | Línea inicial enviada hacia <code>read_column_to_stack</code> .                                                                               |
| x14      | Línea final enviada hacia <code>read_column_to_stack</code> .                                                                                 |
| x15      | Columna recibida como parámetro y validada por el módulo.                                                                                     |
| x18      | Cantidad esperada de datos dentro del rango.                                                                                                  |
| x19      | Línea inicial convertida a entero.                                                                                                            |
| x20      | Valor ideal utilizado para calcular el error.                                                                                                 |
| x21      | Línea final convertida a entero.                                                                                                              |
| x22      | Acumulador de la suma de errores cuadrados.                                                                                                   |
| x23      | Error temporal y error cuadrado durante el ciclo RMSE.                                                                                        |
| x24      | Puntero a los datos cargados en la pila.                                                                                                      |
| x26      | Cantidad real de datos leídos desde el CSV.                                                                                                   |
| x27      | Contador del ciclo principal.                                                                                                                 |
| x28      | Resultado de MSE y posteriormente resultado RMSE.                                                                                             |

| Registro | Uso dentro del módulo                                                                   |
|----------|-----------------------------------------------------------------------------------------|
| x29, x30 | Registros utilizados para preservar marco de pila y dirección de retorno en subrutinas. |

## 4 Ciclos, saltos y subrutinas

El ciclo principal se encuentra en la etiqueta `loop_rmse`. Este ciclo se repite una vez por cada dato leído dentro del rango seleccionado. La cantidad de repeticiones es igual a `COUNT`.

En cada iteración se realiza:

1. Leer un valor desde la pila.
2. Calcular la diferencia entre el valor leído y el valor ideal.
3. Elevar la diferencia al cuadrado.
4. Acumular el error cuadrado.
5. Avanzar al siguiente dato.

### Subrutinas utilizadas

| Subrutina                         | Descripción                                                                                                                                                     |
|-----------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>cadena_a_entero</code>      | Convierte cadenas numéricas recibidas como argumentos a valores enteros. Pertenecce a <code>utils.s</code> .                                                    |
| <code>read_column_to_stack</code> | Lee <code>lecturas.csv</code> , extrae la columna seleccionada dentro del rango solicitado y guarda los valores en la pila. Pertenecce a <code>utils.s</code> . |
| <code>integer_sqrt</code>         | Calcula la raíz cuadrada entera truncada del MSE. Pertenecce a <code>utils.s</code> .                                                                           |
| <code>imprimir_ok</code>          | Imprime la salida estructurada del resultado en consola.                                                                                                        |
| <code>escribir_archivo_ok</code>  | Escribe la salida estructurada en <code>resultado_rmse.txt</code> .                                                                                             |
| <code>write_text</code>           | Realiza escritura por salida estándar mediante <code>sys-call</code> .                                                                                          |
| <code>uint_to_ascii</code>        | Convierte enteros sin signo a texto ASCII para impresión.                                                                                                       |

Los saltos condicionales se utilizan para validar argumentos, rango, columna, cantidad de datos y finalización del ciclo. Entre ellos se emplean `blt`, `bgt`, `beq` y `bne`.

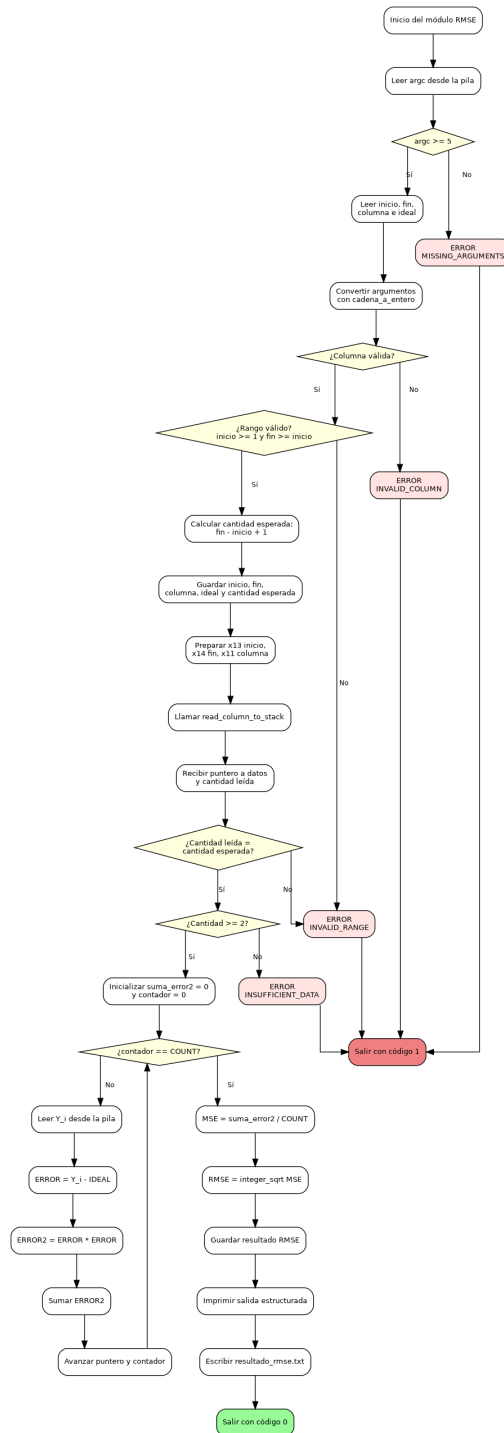


Figura 1: Flujo general de la rutina RMSE.

## 5 Memoria

El módulo utiliza la sección `.data` para almacenar mensajes de salida, mensajes de error y el nombre del archivo de resultado.

La sección `.bss` reserva espacio para variables temporales utilizadas durante la ejecución.

| Variable                         | Uso                                                             |
|----------------------------------|-----------------------------------------------------------------|
| <code>buffer_num</code>          | Buffer para convertir números a texto.                          |
| <code>save_window_start</code>   | Guarda la línea inicial.                                        |
| <code>save_window_end</code>     | Guarda la línea final.                                          |
| <code>save_column</code>         | Guarda la columna seleccionada.                                 |
| <code>save_ideal</code>          | Guarda el valor ideal.                                          |
| <code>save_count</code>          | Guarda la cantidad real de datos procesados.                    |
| <code>save_rmse</code>           | Guarda el resultado final RMSE.                                 |
| <code>save_expected_count</code> | Guarda la cantidad esperada de datos según el rango solicitado. |

Los valores de la columna seleccionada son cargados en la pila por `read_column_to_stack`. El módulo recorre esa zona de memoria avanzando 16 bytes por cada dato.

## 6 Entrada y salida

El formato de entrada del módulo es:

```
modulo_1_rmse inicio fin columna ideal
```

Ejemplo:

```
./build/modulo_1_rmse 4 29 2 25
```

Donde:

- 4: línea inicial.
- 29: línea final.
- 2: columna TEMP.
- 25: valor ideal.

El formato de salida es estructurado:

```
CALC=RMSE
COLUMN=2
WINDOW_START=4
WINDOW_END=29
COUNT=26
IDEAL=25
RMSE=1
STATUS=OK
```

En caso de error, el módulo devuelve una salida estructurada. Por ejemplo:

```
STATUS=ERROR
ERROR=INVALID_COLUMN
DETAIL=COLUMN_MUST_BE_2_TEMP_3_HUM_AIRE_4_SOIL1_6_LUZ_OR_7_GAS
```

## 7 Utilización de utils.s

El módulo utiliza `utils.s` como biblioteca común para reutilizar rutinas de conversión, lectura de datos y cálculo auxiliar.

Las funciones reutilizadas son:

- `cadena_a_entero`: conversión de argumentos a enteros.
- `read_column_to_stack`: lectura de columna y rango desde `lecturas.csv`.
- `integer_sqrt`: cálculo de raíz cuadrada entera.
- `itoa`: conversión de números a texto para escritura en archivo.
- `open_file`: apertura de `resultado_rmse.txt`.
- `write_file`: escritura del resultado en archivo.
- `close_file`: cierre del archivo de resultado.

El cálculo principal RMSE permanece dentro de `modulo_1_rmse.s`. La biblioteca común se utiliza únicamente para operaciones reutilizables de soporte.

## 8 Compilación y ejecución

Para compilar únicamente el módulo RMSE:

```
make build/modulo_1_rmse
```

Para ejecutar el módulo con QEMU:

```
qemu-aarch64 build/modulo_1_rmse 4 29 2 25
```



También puede ejecutarse mediante el objetivo **run** del Makefile:

```
make run MOD=modulo_1_rmse ARGS="4 29 2 25"
```

La salida esperada para este ejemplo es:

```
CALC=RMSE  
COLUMN=2  
WINDOW_START=4  
WINDOW_END=29  
COUNT=26  
IDEAL=25  
RMSE=1  
STATUS=OK
```